

Relay — Technical Design Document (TDD)

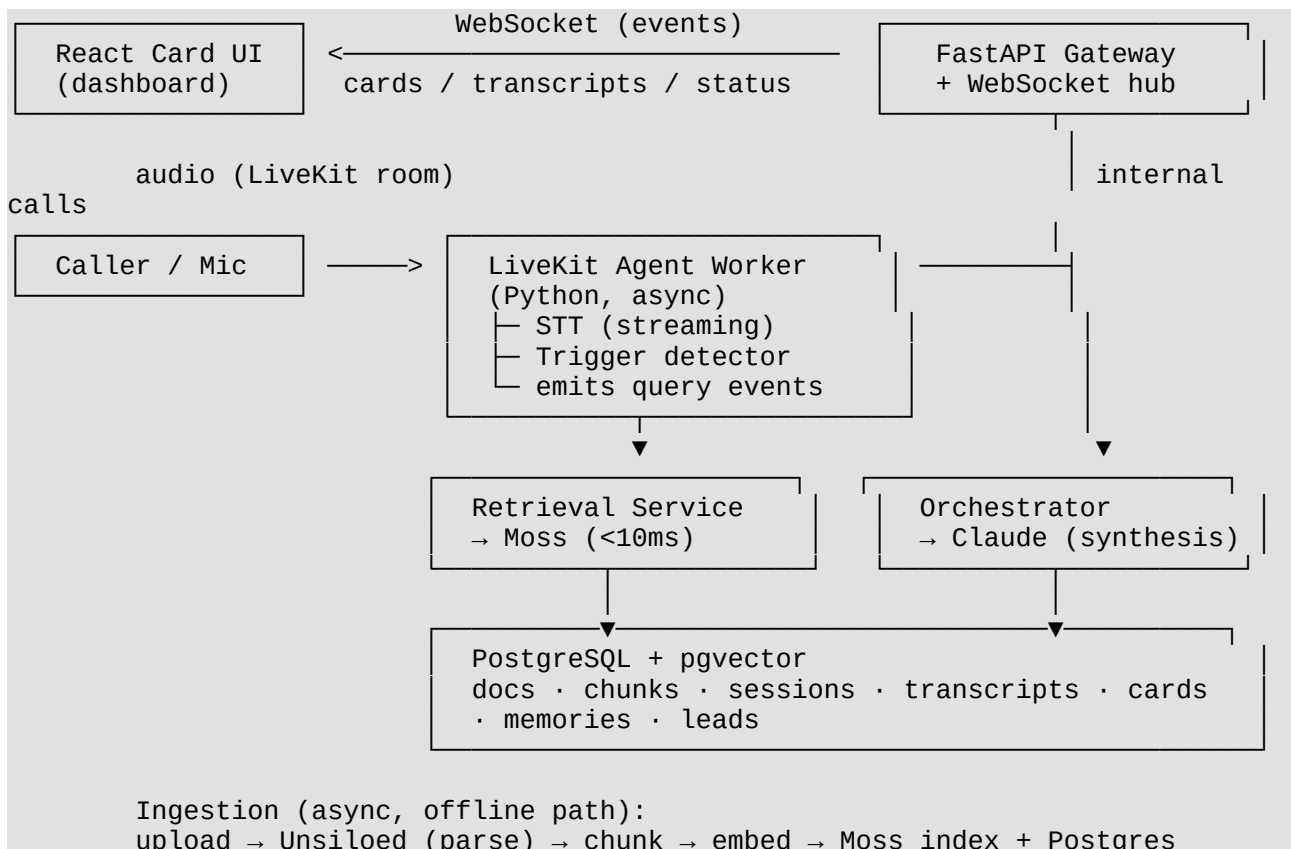
Status: Draft for build · Owner: Systems/Backend · Last updated: June 2026

1. Overview

Relay is a real-time retrieval pipeline. Audio enters a live room, is transcribed, triggers a sub-10 ms retrieval against the org's indexed knowledge, and a synthesized, cited card is pushed to a dashboard. The same pipeline backs all three modes — they differ only in *what triggers retrieval* and *how the result is presented*.

The hard problem is **latency**: a natural reply budget is ~200 ms, but cold vector lookups run 97–307 ms. Moss (<10 ms) is what makes grounded, real-time retrieval feasible. The architecture is built around protecting that budget.

2. Architecture



3. Components

3.1 LiveKit Agent Worker (real-time core)

- Joins the LiveKit room, subscribes to the audio track.
- Streams audio to STT; receives partial + final transcripts.

- Runs the **trigger detector** on the transcript stream.
- On trigger, emits a **query** to the Retrieval Service and forwards transcripts to the gateway for display.
- Language: Python (async), using the LiveKit Agents framework.

3.2 STT (speech-to-text)

- Streaming, low-latency. Candidate: Deepgram / Whisper-class streaming model.
- Emits partial transcripts (for display) and finals (for triggering).
- Configurable endpointing to balance latency vs accuracy.

3.3 Trigger detector

- Decides *when* to fire retrieval. Strategy (configurable):
 - **Question detection:** rule + lightweight classifier on final utterances ("?", interrogatives, intent cues).
 - **Topic/entity shift:** keyword/entity deltas vs the doc index.
 - **Continuous (fallback):** debounced retrieval every N seconds on the rolling window.
- Default for demo: question-detection + debounced continuous, deduped.

3.4 Retrieval Service → Moss

- Receives a query, returns top-k chunks from Moss with scores and source refs.
- Target <10 ms. This is the latency moat; everything else is budgeted around it.
- Falls back to pgvector if Moss is unavailable (slower, demo-safe).

3.5 Orchestrator → Claude

- Takes top-k chunks + the conversation window + mode, produces a **card**: a 1–2 sentence grounded answer plus the source citation.
- Strict prompt: answer *only* from provided chunks; if nothing relevant, return "no card."
- Minimax is an optional secondary model (for a track tie-in / fallback).

3.6 Ingestion Service (offline)

- `upload` → `Unsiload` parses PDFs/docs (tables, layout) → text.
- Chunk (semantic/size-based) → embed → write to Moss index + Postgres (`documents`, `chunks`).
- Idempotent; re-ingest replaces by `document_id`.

3.7 Memory Service

- Cross-session facts keyed by user/customer (Desk + Intake). Postgres + pgvector.

- Demo scope: store + retrieve recent session summaries and key facts.

3.8 Gateway + WebSocket hub (FastAPI)

- REST for ingestion, sessions, documents, leads, manual query.
- WebSocket per session for transcript + card + status events to the dashboard.

3.9 React Card UI

- Connects to the session WebSocket, renders live transcript and cards.
- Mode switcher (Live / Desk / Intake). Card actions: pin, dismiss, expand source.

4. Latency budget (Relay Live)

Stage	Target
STT partial → final	~100–200 ms (model-dependent)
Trigger decision	<5 ms
Moss retrieval	<10 ms
Claude synthesis (short card)	~150–300 ms
WS push + render	<30 ms
Perceived (trigger → card)	<500 ms

Mitigations: stream the card token-by-token; pre-warm Claude connection; cache embeddings of the rolling query window; pre-index demo dataset.

5. Tech stack

- **Audio transport:** LiveKit
- **STT:** streaming (Deepgram / Whisper-class)
- **Retrieval:** Moss (<10 ms)
- **Doc parsing:** Unsiloed
- **LLM:** Claude (primary), Minimax (optional)
- **Backend:** Python 3.11, FastAPI, async; LiveKit Agents
- **DB:** PostgreSQL 15 + pgvector
- **Frontend:** React 18 + TypeScript + WebSocket
- **Deploy:** AWS / TrueFoundry (sponsor credits)
- *(Optional)* **TTS:** Kokoro-class / ElevenLabs for whisper-back

6. Failure modes & fallbacks

- **Moss down** → fall back to pgvector retrieval (slower, still grounded).

- **STT degraded** → show partials only; allow manual typed query.
- **No relevant chunk** → orchestrator returns "no card" (never hallucinate).
- **Demo network risk** → local pre-indexed dataset + recorded backup of the working flow.

7. Security & privacy (noted, out of scope for build)

- Per-org data isolation; cards cite only that org's docs.
- Ambient listening is consent-gated in real deployments (record/notify).
- Secrets via env, never committed (`.env.example` only).

8. Related docs

`API_SPEC.md` · `DATA_MODEL.md` · `ADRS.md` · `SCOPE.md` · `TEST_PLAN.md`